

SIMATS ENGINEERING

COURSE: Artificial Intelligence (CSA17) — ASSESSMENT TOOL 1

I Viswajith (Name / Reg No 192565045) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: 20/07/26

Annexure A: Code of Conduct / Academic Integrity

I Viswajith (Name / Reg No 192565045) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Question 1: Water Jug Problem Solution Path

Problem Formulation: State configuration represented via coordinate pairs (x, y), where x is the 4-Gallon Jug capacity, and y is the 3-Gallon Jug capacity. The targeted goal is to isolate precisely 2 gallons inside the 4-Gallon configuration state: (2, y).

Step	State (x, y)	Action Executed Description
1	(0, 0)	Initial Empty State configuration.
2	(0, 3)	Fill the 3-Gallon Jug completely from the pump supply.
3	(3, 0)	Pour contents entirely from the 3-Gallon Jug into the 4-Gallon unit.
4	(3, 3)	Refill the 3-Gallon Jug completely from the pump supply.
5	(4, 2)	Pour water into the 4-Gallon Jug until completely full (leaves exactly 2g inside the 3-Gallon container).
6	(0, 2)	Empty the 4-Gallon Jug completely directly onto the ground.
7	(2, 0)	Transfer the remaining 2 gallons from the 3-Gallon container into the empty 4-Gallon Jug. [Goal Reached]

Question 2: Mars Rover Environment Formulation (PEAS)

Percepts: Stereoscopic visual feeds, LiDAR point-cloud maps, infrared spectroscopy data, atmospheric thermal/wind indexes, chassis pitch/yaw vectors, and mechanical load cells.

Environment: Partially Observable (limited sensor field-of-view), Stochastic (unpredictable atmospheric dynamics), Sequential (actions impact residual battery constraints), Dynamic, Continuous, and Single-Agent.

Actions: Angular wheel steering adjustments, progressive velocity drive acceleration, deployment of core drills/scoops, internal lab thermal chamber handling, and directional data-link positioning.

Performance Metrics: Volumetric density of specialized rock materials sampled and characterized, prevention of hazardous telemetry states, and battery management efficiency.

Optimal Architecture: A *Utility-Based Hybrid Agent Framework* is selected. This provides internal evaluation of alternative route structures under multi-tier destination models while independently navigating round-trip latencies of solar cycles without crashing.

Question 3: 8-Queens Search Formulation Strategy

Incremental Search Tree Strategy: Begins with an empty chessboard grid matrix. Systematically places one individual queen per column array sequentially while immediately verifying conflicts down row, column, and cross-diagonal matrices. Reduces state evaluation space to a calculated max limit permutation configuration bounding of $8!$ (40,320 distinct states).

Complete-State Formulation (Local Heuristic Search): Configures all 8 queens onto the chessboard layout concurrently (one assigned per column). An evaluation calculation heuristic metric, h , tracks the exact number of mutual queen threat pairs. Optimization engines execute Hill-Climbing adjustments to reduce the conflict heuristic calculation until an absolute minimal convergence mapping where $h = 0$ is discovered.

Question 4: OLA Cab Booking Problem-Solving Agent Structure

The application architecture operates under a **Goal-Based Problem-Solving Agent model** using an explicit Constraint-Satisfaction design tracking user preferences.

```
function SOLVE-CAB-ROUTING(origin, destination, user_preference) returns action_sequence
  problem.INITIAL_STATE ← origin; problem.GOAL_STATE ← destination
  eligible_drivers ← FILTER_BY_TIER(GET_ACTIVE_DRIVERS_NEARBY(origin), user_preference)
  if eligible_drivers is empty then return failure
  selected_driver ← FIND_MINIMUM_ETA(eligible_drivers, origin)
  action_sequence ← SEARCH_OPTIMAL_PATH(LOAD_LIVE_TRAFFIC_MAP(), origin, destination)
  execute DISPATCH_CAB(selected_driver, origin)
  return action_sequence
```

Question 5: Delivery Network Route Optimization (Uniform Cost Search)

Uniform Cost Search (UCS) determines the cheapest path based strictly on cumulative edge cost tracking using a priority queue. Below is the evaluation trace sequence mapping minimum total expenditure steps toward the goal node destination 'G':

Iteration Step	Node Visited	Priority Queue State (Sorted Path Cost)	Explored Set Update
Init Step	-	[(0, 'S', ['S'])]	{}
Step 1	S	[(1, 'A', ['S','A']), (12, 'G', ['S','G'])]	{S}
Step 2	A	[(2, 'C', ['S','A','C']), (4, 'B', ['S','A','B']), (12, 'G', ['S','G'])]	{S, A}
Step 3	C	[(3, 'D', ['S','A','C','D']), (4, 'B', ['S','A','B']), (4, 'G', ['S','A','C','G']), (12, 'G', ['S','G'])]	{S, A, C}
Step 4	D	[(4, 'B', ['S','A','B']), (4, 'G', ['S','A','C','G']), (6, 'G', ['S','A','C','D','G']), (12, 'G', ['S','G'])]	{S, A, C, D}
Step 5	B	[(4, 'G', ['S','A','C','G']), (6, 'G', ['S','A','C','D','G'])]	{S, A, C, D, B}
Step 6	G	Goal Node reached via popped queue state. Termination active.	{S, A, C, D, B, G}

Optimal Transportation Target Route Resolved: S → A → C → G
Minimum Feasible Network Path Execution Expenditure Cost: 4 Units